

ECE-3103

8086 Interrupt

Md. Ebtidaul Karim

Introduction

- Most Microprocessors allow normal program execution to be interrupted by some external signal or by a special instruction in the program.
- Interrupts are events occurring asynchronously/synchronously sending signals to the CPU to inform their occurrence and for the CPU to take appropriate action.
- In response to an interrupt, the microprocessor stops executing it's current program and call a procedure known as ISR(Interrupt Service Routine) which serves the interrupt.

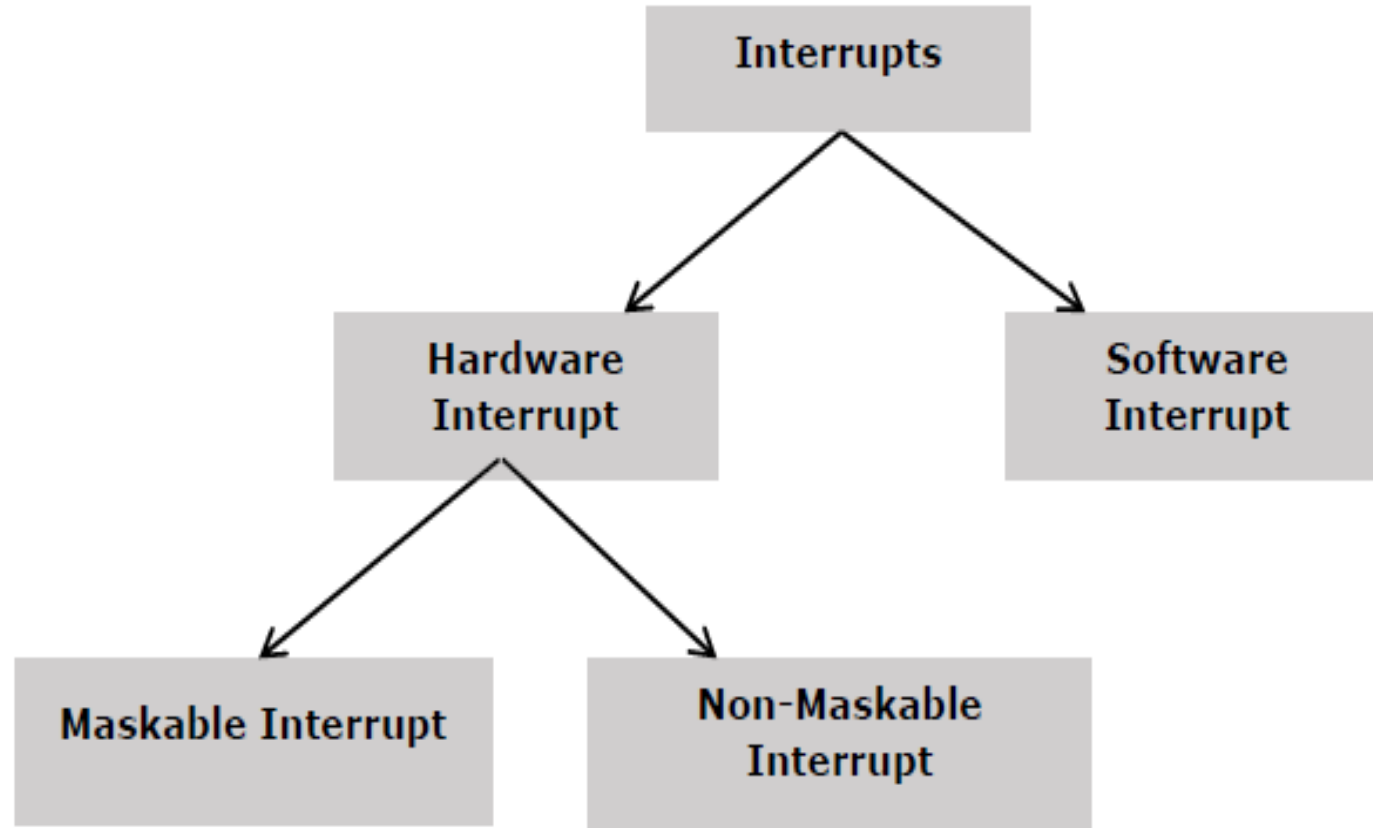
Introduction

- ISR is a program that tells the processor what to do when the interrupt occurs.
- After the execution of ISR, control returns back to the main routine where it was interrupted.

An 8086 interrupt can come from any one of the **three sources**.

1. External signal applied to NMI pin or INTR input pin Hardware interrupt
2. Execution of Interrupt instruction, INT software interrupt
3. Some error condition produced by executing an instruction i.e. divide by zero

Sources



Hardware Interrupt

- Hardware interrupt is caused by any peripheral device by sending a signal through a specified pin to the microprocessor.
- The 8086 has two hardware interrupt pins, i.e. NMI and INTR.
- NMI is a non-maskable interrupt and INTR is a maskable interrupt having lower priority.
- One more interrupt pin associated is INTA called interrupt acknowledge.
- Example of maskable interrupts are signals from I/O devices indicating either its readiness or completion of I/O, etc.
- Examples of NMI are signals from a nuclear reactor or any controlled system that is being controlled by the CPU.

Hardware Interrupt

- When either a masked or non-maskable interrupt comes, the CPU completes the instruction of the main program it is currently executing and then executes a program written for that interrupt which is called Interrupt Service Routine(ISR).
- After executing the ISR, if no further interrupts are there, the CPU returns to the next instruction in the main program which it was to execute, if the interrupt had not occurred.
- Since the CPU has to return to the next instruction of the main program it is executing after execution of ISR, the CPU has to save the statuses of some registers such as program counter, accumulator, conditional code register etc, corresponding to the main program before taking up the ISR.

Hardware Interrupt

- Saving of these essential registers, called current CPU status, is done by special hardware and not by executing machine instructions.
- To start executing ISR, the address of the first machine instruction of ISR has to be loaded into the Program Counter(PC).
- This address is called **interrupt vector**.
- The time that the CPU takes from the moment it accepts the interrupt to the moment it starts executing the ISR is called **interrupt response time**.

Software Interrupt

- They are synchronous and generated by CPU.
- These are instructions that are inserted within the program to generate interrupts.
- There are 256 software interrupts in 8086 microprocessor.
- The instructions are of the format INT type where type ranges from 00 to FF.
- **Here the ISR processing is similar to that of hardware interrupt.**

Exceptions

❑ **Faults:**

- ✓ If there is any problem in writing instruction.
- ✓ If we try to access any restricted memory location of microprocessor or memory location that is not present in microprocessor(not within the range of 00000H-FFFFFFH).

❑ **Traps:**

- ✓ Invalid Opcode.
- ✓ Divide by Zero.
- ✓ Parity Failure.
- ✓ Overflow.

Exceptions

- ❑ Abort: If there is any problem in hardware.
- ✓ Here CPU cannot proceed executing the main program, and so it aborts the main program immediately and flashes a message to the operator.
- ❑ There are also some other interrupts like power off. The CPU on occurrence of interrupt, completes the execution of current instruction, saves CPU status in a non-volatile storage and shuts down.
- ❑ When the power returns, the CPU can restore the status of the CPU at the time of power off and continue to execute the program that got suspended due to power off.

Interrupt Response

- At the end of the each instruction cycle, the 8086 checks to see if any interrupts have been requested.
- If an interrupt has been requested, the 8086 responds to the interrupt by stepping through the following series of major actions:
 1. It decrements the SP by 2 and pushes the flag register on the stack.
 2. It disables the 8086 INTR interrupt input by clearing the interrupt flag (IF) in the flag register.
 3. It resets the trap flag (TF) in the flag register.

Interrupt Response

5. It decrements the SP by 2 and pushes the current CS register contents on stack.
6. It decrements the SP again by 2 and pushes the current instruction pointer contents on the stack.
7. It does an indirect far jump to the start of the procedure you wrote to respond to the interrupt.

Interrupt Response

- ❑ An **IRET** instruction at the end of the interrupt service procedure returns execution to the main program.

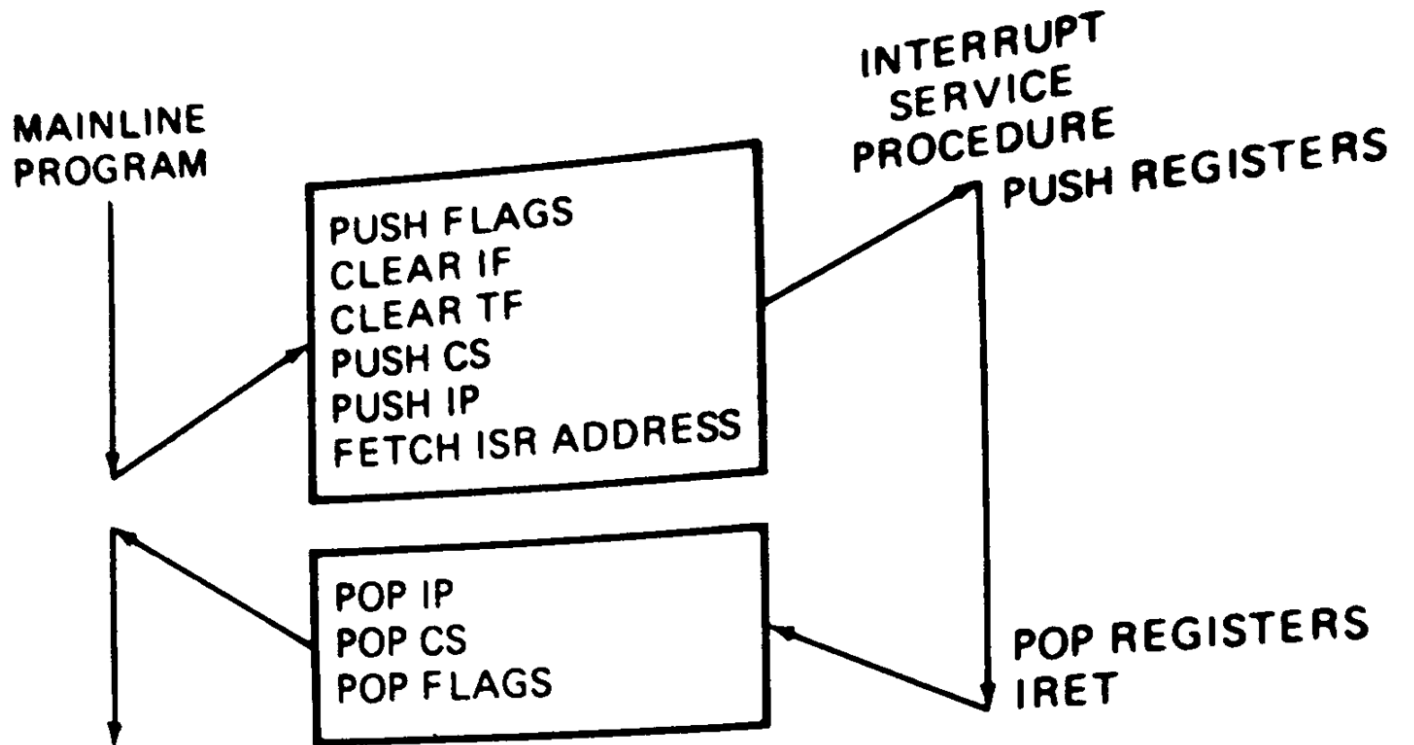


FIGURE 8-1 8086 interrupt response.

Interrupt Pointer Table

- When the 8086 responds to an interrupt, as it is like an indirect far CALL it goes to four memory locations to get the new CS and IP values for the start of the interrupt service procedure.
- In an 8086 system, the first 1Kbyte of memory, from 00000H to 003FFH is set aside as a table for storing the starting addresses of interrupt service procedure.
- Since 4 bytes are required to store the CS and IP values for each interrupt service procedure, the table can hold the starting addresses for up to $1024/4 = 256(0-255)$ interrupt procedures.
- The starting address of an interrupt service procedure is often called the **interrupt vector** or the **interrupt pointer** and the table is referred to as the **interrupt vector table** or the **interrupt pointer table**.

Interrupt Pointer Table

- ✓ IP value is put in as the low word of the vector, and the CS register is put as the high word of the vector.
- ✓ Each double word interrupt vector is defined by a number from 0 to 255, commonly known as **type**.
- ✓ The lowest five types are dedicated to specific interrupts, such as divide by zero interrupt etc.
- ✓ Interrupt types 5 to 31 are reserved by Intel for use in more complex microprocessors, such as 80286, 80386 and 80486.
- ✓ The upper 224 interrupt types are available to use for hardware or software interrupt.

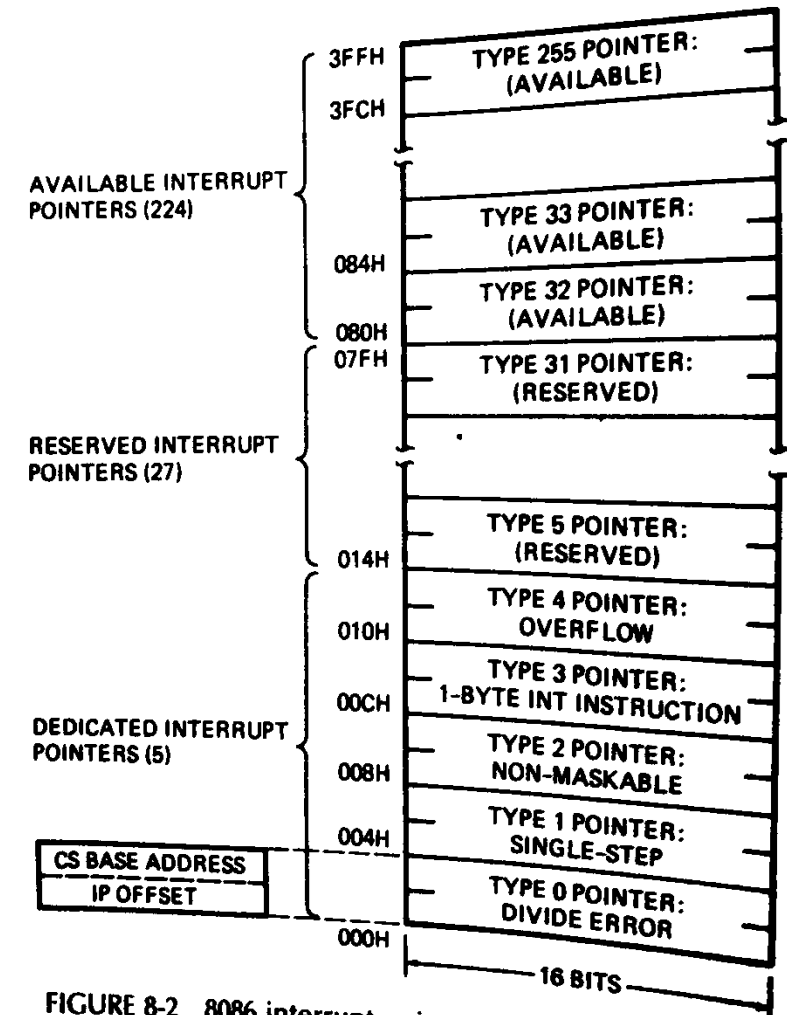


FIGURE 8-2 8086 interrupt-pointer table.

Interrupt Pointer Table

- ✓ Vector for each interrupt type requires four memory locations.
- ✓ Therefore, when the 8086 responds to a particular type interrupt, it automatically multiplies the type by 4 to produce the desired address of IP in the vector table and CS address will be the next word.
- ✓ For example starting address of type 0 interrupt will be $00000H * 4 = 00000H$ and its CS address is $00000H + 2 = 00002H$
- ✓ Starting address of type 3 interrupt is $00003H * 4 = 0000CH$ and its CS address is $0000CH + 2 = 0000EH$.

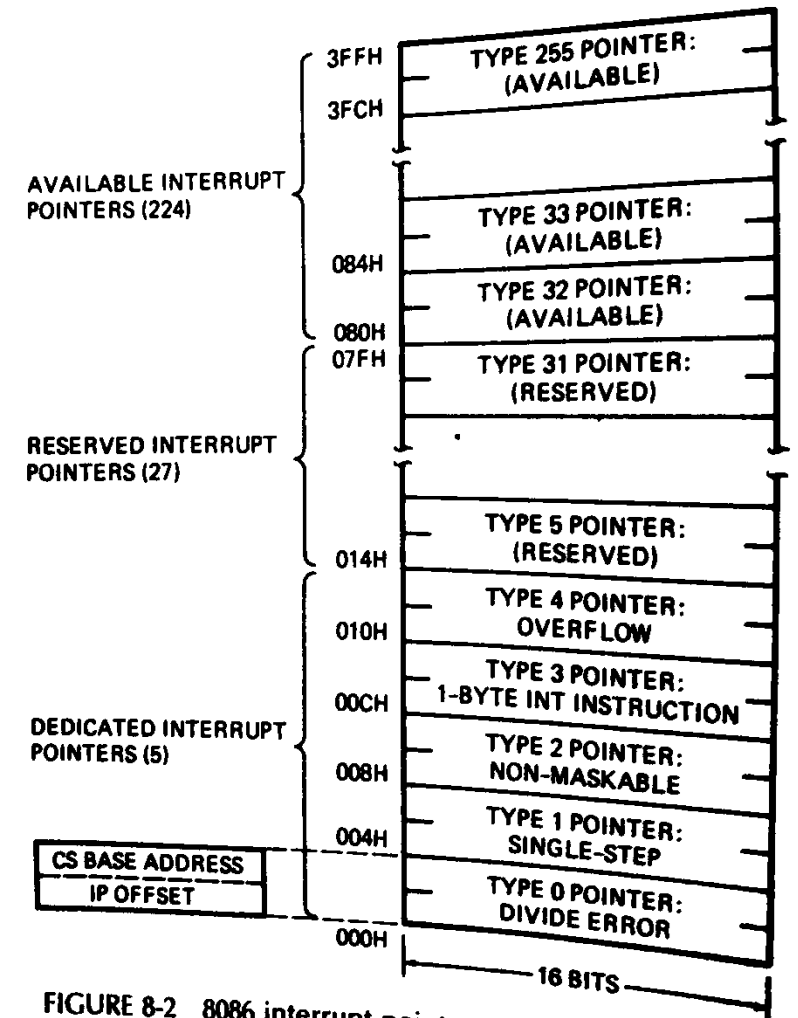


FIGURE 8-2 8086 interrupt-pointer table.

Type 0 Interrupt

- DIV, IDIV instruction allows us to divide a 16 bit binary number in AX by a 8 bit number, as a result 8 bit result (quotient) from division will be left in the AL register and 8 bit remainder will be left in the AH register.
- Similarly DIV, IDIV instruction allows us to divide a 32 bit binary number in DX and AX by a 16 bit number, as a result 16 bit result (quotient) from division will be left in the AX register and 16 bit remainder will be left in the DX register.
- Whenever the quotient from a DIV or IDIV operation is too large to fit in the result register, the 8086 will automatically do a type 0 interrupt.

Type 0 Interrupt

- In response to this interrupt the 8086 proceeds as follow:
 1. It decrements the SP by 2 and pushes the flag register on the stack.
 2. It disables the 8086 INTR interrupt input by clearing the interrupt flag (IF) in the flag register.
 3. It resets the trap flag (TF) in the flag register.
 5. It decrements the SP by 2 and pushes the current CS register contents on stack.
 6. It decrements the SP again by 2 and pushes the current instruction pointer contents on the stack.
 7. 8086 then gets the starting address of the interrupt service procedure from the type 0 locations in the interrupt-vector table.

Type 0 Interrupt

8. It gets the new value for CS from addresses 00002H and 00003H and new value for IP from addresses 00000H and 00001H.
9. After the starting addresses of the procedure is loaded into CS and IP, the 8086 then fetches and executes the first instruction of the procedure.
10. At the end of the interrupt service procedure, an IRET instruction is used to return execution to the interrupted program.
11. It then POPS the stored IP off the stack and increments the SP by 2, then POPS the stored CS off the stack and increments the SP by 2 and finally restores flag by POP Flag off the stack and increments SP by 2.
12. In short IRET returns execution to the interrupted program and restores IF and TF to the state they were in before the interrupt.

Type 1 Interrupt

- If the 8086 trap flag is set, the 8086 will automatically do a type 1 interrupt after each instruction executes.
- When the 8086 does a type 1 interrupt, it pushes the flag register on the stack, resets TF and IF, and pushes the CS and IP values for the next instruction on the stack.
- It then gets the IP value for the start of the type 1 interrupt-service procedure from address $00001H * 4 = 00004H$ and it gets the CS value for the start of the procedure from $00004H + 2 = 00006H$ (From next word).

Type 2 Interrupt or NMI

- The 8086 will automatically do a type 2 interrupt response when it receives a low to high transition on its NMI input pin.
- When this interrupt is activated, these actions take place –
- Completes the current instruction that is in progress.
- Pushes the Flag register values on to the stack.
- Interrupt flag and trap flag are reset to 0.
- Pushes the CS (code segment) value and IP (instruction pointer) value of the return address on to the stack.

Type 2 Interrupt or NMI

- It then gets the IP value for the start of the type 2 interrupt-service procedure from address $00002H * 4 = 00008H$ and it gets the CS value for the start of the procedure from $00008H + 2 = 0000AH$ (From next word).
- Type 2 interrupt response cannot be disabled (masked) by any program instructions.
- Since this input cannot be intentionally or accidentally disabled we use it to signal the 8086 that some condition in an external system must be taken care of.
- We also use this interrupt to save program data in case of a power failure.

Type 3 interrupt or Breakpoint Interrupt

- The type 3 interrupt is produced by execution of the INT 3 instruction.
- It is used to implement a breakpoint function in a system.
- It is a 1-byte instruction having op-code is CCH.
- These instructions are inserted into the program so that when the processor reaches there, then it stops the normal execution of program and follows the break-point procedure and then waits for the next command from the user.
- Its execution includes the following steps –
- Flag register value is pushed on to the stack.

Type 3 interrupt or Breakpoint Interrupt

- Interrupt Flag and Trap Flag are reset to 0
- CS value of the return address and IP value of the return address are pushed on to the stack.
- IP is loaded from the contents of the word location $3 \times 4 = 0000CH$
- CS is loaded from the contents of the next word location $0000CH + 2 = 0000EH$.

Overflow Interrupt - Type 4

- Two major ways to detect and respond to an overflow error.
- INTO (interrupt on overflow), immediately after the arithmetic instruction.
- It is a 1-byte instruction and their mnemonic **INTO**. The op-code for this instruction is CEH.
- If the overflow flag is not set when the 8086 executes the INTO instruction, the instruction will simply function as NOP.
- However If the overflow flag is set, indicating an overflow error, the 8086 will do a type 4 interrupt after it executes INTO instruction.

Overflow Interrupt - Type 4

- When 8086 executes type 4 interrupt it follows below steps:
- Flag register values are pushed on to the stack.
- Interrupt flag and Trap flag are reset to 0
- CS value of the return address and IP value of the return address are pushed on to the stack.
- IP is loaded from the contents of word location $00004H * 4 = 00010H$
- CS is loaded from the contents of the next word location 00012H.

Overflow Interrupt - Type 4

- Another way to detect and respond to an overflow error in a program is to put the jump if overflow instruction (JO) immediately after the arithmetic instruction.
- If the overflow flag is set as a result of arithmetic operation, execution will jump to the address specified in the JO instruction.
- At this address, we can put an error routine which responds in the way you want to the overflow.

Software Interrupt

- The 8086 INT instruction can be used to cause the 8086 to do any one of the 256 possible interrupt types.
- INT- Interrupt instruction with type number. It is 2-byte instruction. First byte provides the op-code and the second byte provides the interrupt type number.
- Its execution includes the following steps –
 - Flag register value is pushed on to the stack.
 - Interrupt Flag and Trap Flag are reset to 0.
 - CS value of the return address and IP value of the return address are pushed on to the stack.

Software Interrupt

- IP is loaded from the contents of the word location 'type number' $\times 4$
- CS is loaded from the contents of the next word location.
- Software interrupts produced by the INT instruction have many uses.
- The IBM PC has in its ROM collection of routines, each performing some specific function such as reading character from keyboard, writing character to CRT.
- This collection of routines referred to as **Basic Input Output System or BIOS**.
- The BIOS routines are called with INT instructions.

Software Interrupt

- Again, we use INT 3 instruction to insert breakpoints in programs for debugging.
- Another use of software interrupts is to test various interrupt-service procedures.
- For example, we can use an INT 0 instruction to send execution to a divide-by-zero interrupt-service procedure without having to run the actual division program.

INTR Interrupts – Type 0 -255

- Unlike the external interrupt NMI, INTR can be masked (disabled) so that it cannot cause an interrupt.
- If the IF flag is cleared then the INTR input is disabled.
- IF flag can be cleared using CLI (*Clear interrupt*) instruction.
- To enable INTR we have to set IF flag with STI (*Set Interrupt*).
- When the 8086 is reset, the IF is automatically cleared.
- **IF (Interrupt Flag)** is automatically cleared as part of the response of an 8086 to an interrupt.
 - Two reasons

INTR Interrupts – Type 0 -255

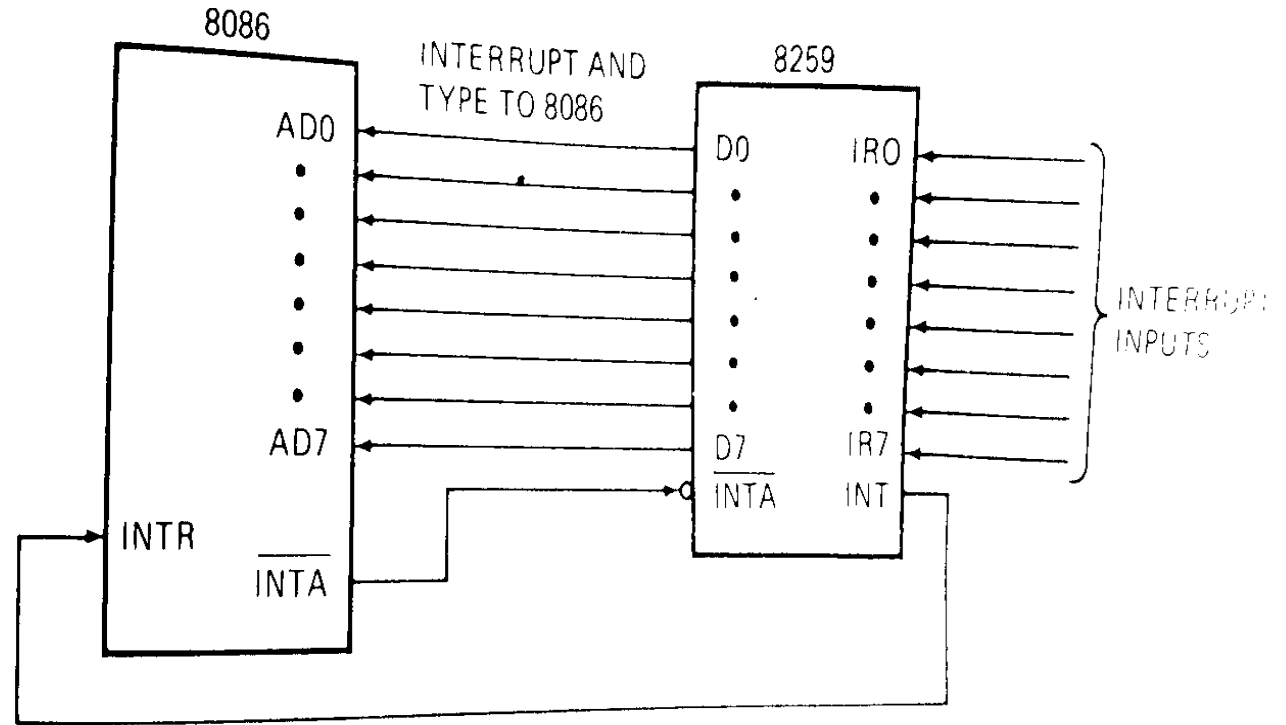
- **First**, it prevents a signal on the INTR input from interrupting a higher-priority interrupt service procedure in progress.
- However if we want another INTR input signal to be able to interrupt an interrupt procedure in progress, we can re-enable the INTR input with an STI instruction at any time.
- The **second** reason is to make sure that a signal on the INTR input does not cause the 8086 to interrupt itself continuously.
- If INTR were not disabled during the first response, the 8086 would be continuously interrupted and would never get to the actual interrupt-service procedure.

INTR Interrupts – Type 0 -255

- The INTR interrupt is activated by an I/O port.
- If the interrupt is enabled and NMI is disabled, then the microprocessor first completes the current execution and sends '0' on INTA pin twice.
- The first '0' means INTA informs the external device to get ready and during the second '0' the microprocessor receives the 8 bit, say X, from the 8259A priority interrupt controller.

INTR Interrupts – Type 0 -255

- When an 8259A receives an interrupt signal on one of its IR input, it sends an interrupt request signal to the INTR input of the 8086.
- If the INTR input of the 8086 has been enabled with an STI instruction, it will respond in the following way.



Block diagram showing an 8259 connected to an 8086.

INTR Interrupts – Type 0 -255

- The 8086 responds to an INTR interrupt as follows :
- The 8086 first does two interrupt acknowledge machine cycles.
- In the first interrupt acknowledge machine cycle the 8086 floats the data bus lines AD0-AD15 and sends out an $\overline{INTA}$ pulse on its $\overline{INTA}$ output pin.
- This indicates an interrupt acknowledge cycle in progress and the system is ready to accept the interrupt type from the external device.

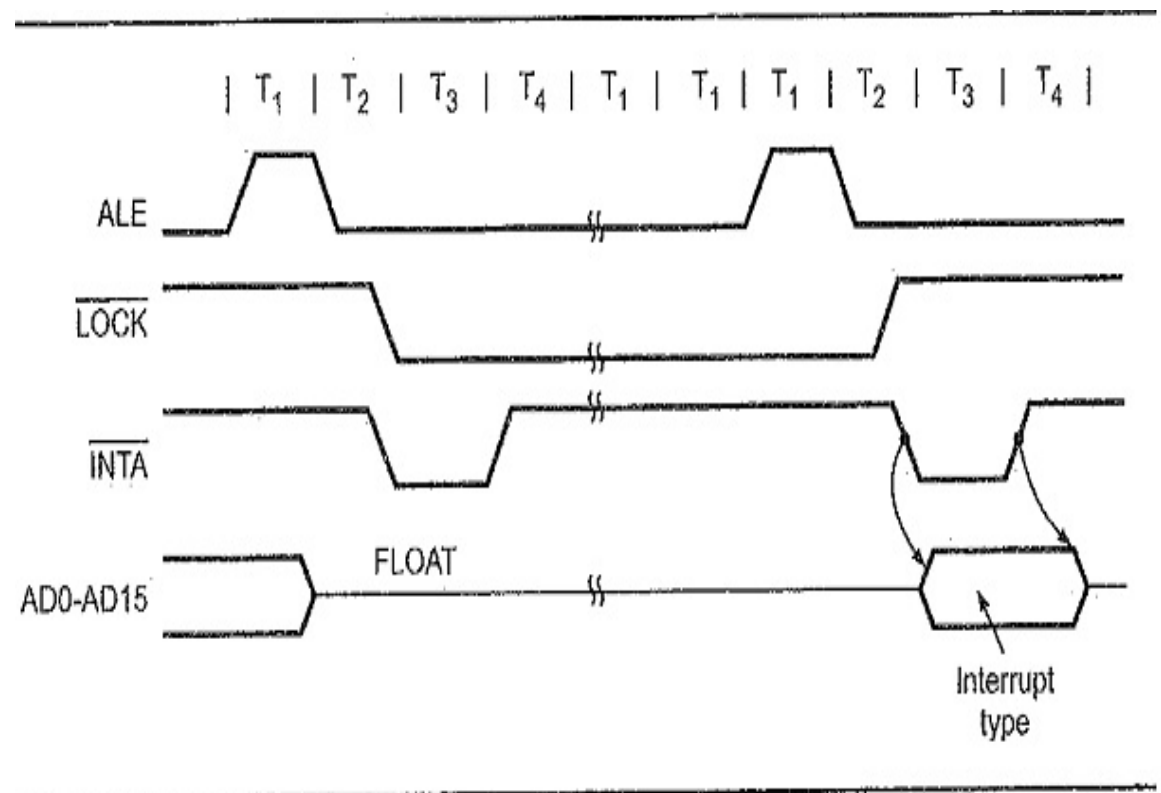


Fig. 9.3 Interrupt acknowledge machine cycle

INTR Interrupts – Type 0 -255

- During the second interrupt acknowledge machine cycle the 8086 sends out another pulse on its $\overline{INTA}$ output pin.
- In response to this second $\overline{INTA}$ pulse the external device puts the interrupt type on lower 8 bits of the data bus.

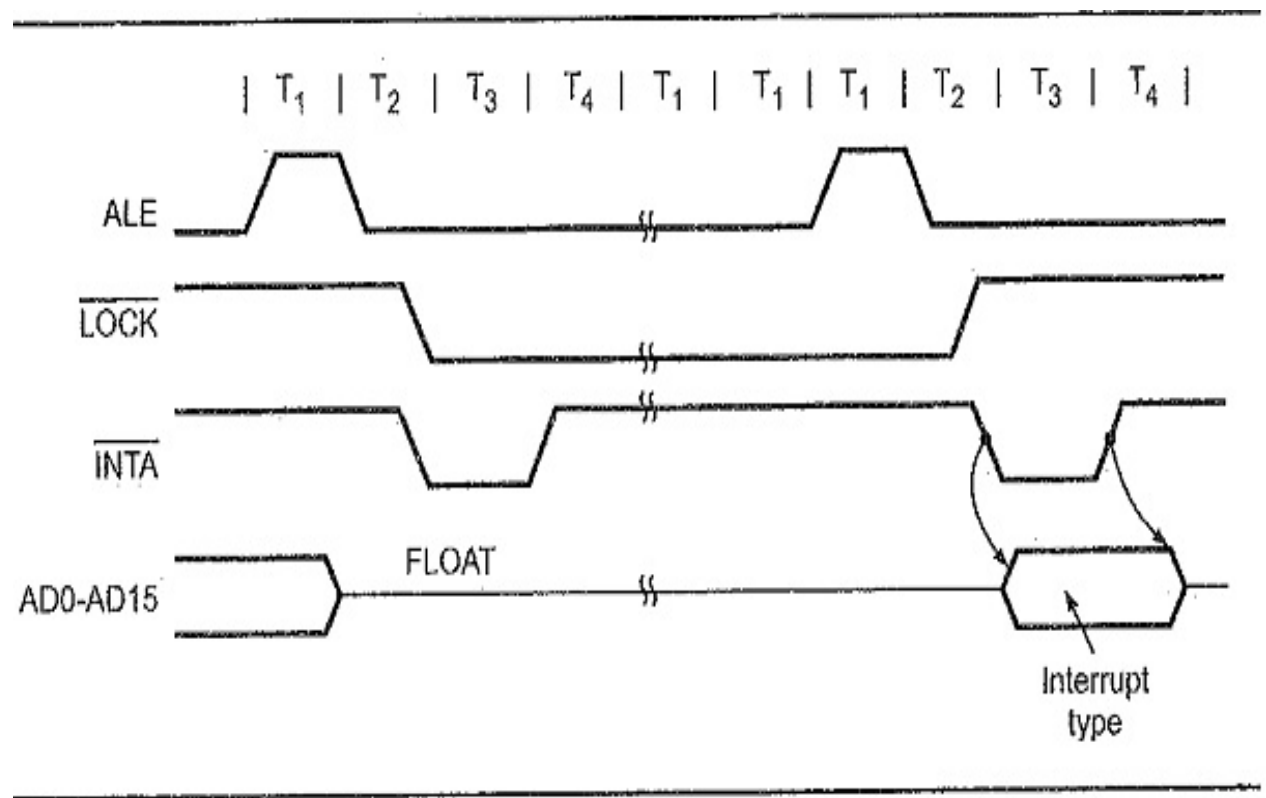


Fig. 9.3 Interrupt acknowledge machine cycle

INTR Interrupts – Type 0 -255

- Once the 8086 receives the interrupt type, it pushes the flag register on the stack, clears TF and IF, and pushes the CS and IP values of the next instruction (return address) on the stack.
- The 8086 then gets the new value of IP from the memory address equal to 4 times the interrupt type (number), and CS value from memory address equal to 4 times the interrupt number plus 2.

8086 Interrupt Priority

- As far as the 8086 Interrupt Priority are concerned, software interrupts (All interrupts except single step, NMI and INTR interrupts) have the highest priority, followed by NMI followed by INTR. Single step has the least priority.

Interrupt	Priority
Divide Error, Int n, Int 0	HIGHEST
NMI	↓
INTR	↓
SINGLE - STEP	LOWEST

8086 Interrupt Priority

- The interrupt flag is automatically cleared as part of the response of an 8086 to an interrupt.
- This prevents a signal on the INTR input from interrupting a higher priority interrupt service routine.
- The 8086 allows NMI input to interrupt higher priority interrupt, for example suppose that a rising edge signal arrives at the NMI input while the 8086 is executing a DIV instruction, and that the division operation produces a divide error.
- Since the 8086 checks for internal interrupts before it checks for an NMI interrupt, the 8086 will push the flags on the stack, clear TF and IF, push the return address on the stack, and go to the start of the divide error service routine.

8086 Interrupt Priority

- The 8086 will then do an NMI interrupt response and execute non-maskable interrupt service routine.
- After completion of NMI service routine an 8086 will return to the divide error routine.
- It will execute divide error routine and then it will return to the main program

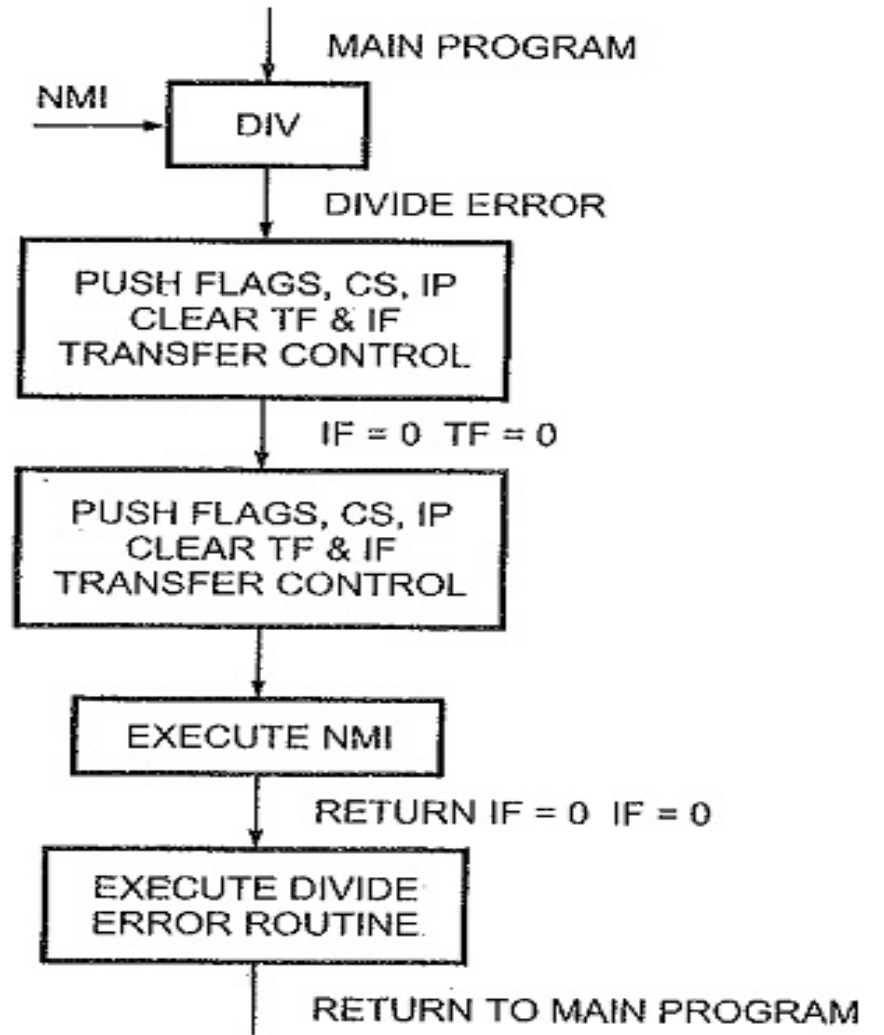


Fig. 9.4 Flow-chart for divide error routine

Thank You